Crossing Queues with Global Re-Ordering for Multi-Forklift Traffic on Free-Form Routes

[TODO: Author names]
[TODO: Affiliation]
[TODO: e-mail]

***Abstract*—Autonomous forklifts on warehouse floors without a fixed roadmap follow free-form, curvature-constrained routes that touch each other at places nobody chose in advance. A traffic layer must decide, at every such place, which truck goes first, without collisions or circular waits, while pedestrians, vehicle faults, spilled loads, message loss and localization error keep invalidating the timing it planned with. We present *crossing queues*: every pair of touching route intervals is a crossing, every truck's interval in it a queue member with its own ticket, time window and token, and a member that misses its window rejoins the line at its new free-flow arrival time wherever this keeps the precedence graph free of cycles. On top of the queues, a mixed-integer linear program re-orders all still-open crossings every 5 s to minimize age-weighted delay, and a speed cap lets a truck arrive as the place clears. In 117 simulated cells on procedurally generated rack, process and multi-hall floors with 5–20 trucks and six disturbance conditions, the full method matched the throughput of a time-window baseline (+0.2 %) while reducing waits longer than 60 s from 343 to 51 and the longest wait from 540 s to 124 s, with no truck–truck overlap and no deadlock. On 1.5×-scaled floors with 20 trucks and chained random jobs it completed 8.3 % [1.9, 15.7] more jobs than the baseline and was statistically indistinguishable from an adapted critical-section coordinator, but its MILP did not keep up with real time on the larger layouts and two cells collapsed after the crossing orders locked.**

***Index Terms*—Multi-robot coordination, AGV traffic management, deadlock avoidance, mixed-integer programming, warehouse logistics.**

I. Introduction

Fleets of autonomous forklifts are moving into warehouses and plants whose layouts change and that have no guide paths. When each truck plans its own curvature-constrained route, e.g., with Hybrid A* [1], the routes touch at places that are not known in advance: junctions three routes meet at, long shared lanes, dead-end aisles a truck enters and leaves again, and reverse segments. A traffic layer then has to decide, at every such place, which truck goes first. The decision must keep trucks apart, must never create a circular wait, and should keep trucks moving, while pedestrians, vehicle faults, spilled loads, message loss and localization error keep invalidating any timing it planned with.

Existing approaches differ in what they reserve and how they order. Zone- and tile-reservation schemes grant space step by step and avoid deadlock with look-ahead checks [2]–[4]. Critical-section coordinators compute where spatial envelopes overlap and assign precedence with a heuristic revised every control period [5]–[7]. Plan-execution graphs keep the passing order a multi-agent plan implies and are robust to delay [8]–[10]; switchable action dependency graphs re-order such a plan with a MILP on a roadmap [11]. Lifelong multi-agent path finding (MAPF) replans space–time reservations in a rolling window [12]–[14].

Our routes are given and continuous, so we keep them and decide only the *order in time* at the places where they touch. This paper asks how much that order is worth and what it costs. Its contributions are:

1) *Crossing queues* over free-form routes: crossings derived from touching route samples, one queue member per truck and crossing with a ticket, a time window and a token, and re-queuing of a late member at its new free-flow time subject to a lock check (Sec. IV).
2) A *global re-ordering* MILP over all open crossings with an age-weighted delay objective, accepted only when the new orders are lock-free and lower the planned delay, together with release-time pacing and an optional load-dependent physical time model (Sec. IV-A).
3) An evaluation whose parameters were fixed on development seeds before the evaluation seeds were run once: 117 cells on generated floors under six disturbance conditions against six other policies, three of them literature methods adapted to the same safety layer, and a 36-cell large-scale study with paired bootstrap intervals that also reports where the method fails (Secs. V–VI).

II. Related Work

Resource reservation. Fransen et al. [2] tessellate the floor into tiles of any shape and grant a tile reservation only if the resulting state admits a deadlock-free order of single steps within a look-ahead; the paper gives worked examples but no tile size or throughput study. Hierarchical and improved dynamic reservation schemes extend the idea to industrial layouts [3], [4], [15]. Reveliotis studied MPC traffic scheduling and liveness for zone-controlled guidepath systems [16], [17].

Critical sections and precedence. The loosely-coupled coordinator of Pecora et al. [5] computes critical sections where two envelopes overlap, sends each robot a critical point it must not pass, and orders robots heuristically (e.g., by distance to the section); later work proves safety under unreliable communication [6] and liveness with online goals and global re-ordering [7]. Our setting shares the critical-point idea: a truck receives only the end of its grant.

Execution graphs. Temporal plan graphs [8] and action dependency graphs (ADG) [9] execute a MAPF plan by keeping its passing order, which tolerates arbitrary delays but never re-orders; RMTRACK [10] does the same on continuous trajectories. Berndt et al. [11] make the dependencies switchable and re-order them by a MILP in a receding horizon; every feasible solution is acyclic. Our global ordering follows the same idea on route-derived crossings, with tokens and expiry, and without a roadmap. Ordering by alternative graphs in train rescheduling [18], [19] is a further ancestor.

Lifelong MAPF. RHCR [12] resolves collisions only within a window and replans periodically; RHCRE executes it with an ADG [13]. Prioritized planning with safe intervals [20], [21] and fixed-path velocity tuning [22] are related. Recent CASE papers apply lifelong MAPF in plants [14], [23], schedule fleets online around executing robots [24], and combine reciprocal avoidance with local MAPF [25]. These assume a grid or roadmap; we compare against adaptations of three of these families on our fixed routes.

III. System and Problem

A. Fleet, jobs and routes

A fleet of N car-like forklifts (1.46×0.63 m, 24 kg, measured small-scale chassis) serves continuous pallet jobs: from a holding slot, approach and insert into a pallet, lift, extract, carry it to a drop slot, lower, back out and return to a free holding slot. Each stage has a cruise speed (approach 0.60, carry 0.30, extract 0.18, back-out 0.12, insert 0.055 m/s); lifting and lowering take 4.5 s each. Routes are planned per truck in space only, with Hybrid A* [1], and sampled every 0.1 m including reverse segments; all pallet and holding slots other than the truck's own are inflated planning obstacles, so that a parked truck or a pallet placed later never lies on an earlier route.

B. Safety layer (common to all policies)

A central traffic manager grants each truck n a stretch of its route ahead of its progress p_n, up to a look-ahead of $L = 3$ m. The *claim* of n is the body swept over that stretch, inflated by a margin of 0.3 m; a grant is not extended into another truck's body or claim, and the truck brakes to stop at its grant end. This layer alone keeps trucks apart if they stay within their grants; the margin covers tracking and localization error. Two recovery actions run on top of every policy: a detour around a standing truck (TA, after 20 s blocked) and a back-off or detour that treats the other truck's route as a wall (RA, on a wait cycle or after 60 s blocked) [TODO: cite or describe internal TA/RA design].

C. Crossings and the order condition

What the policies decide is who goes first where routes touch. For each pair of trucks (a, b), the remaining routes are sampled every 0.2 m with the body inflated by the margin; touching sample pairs are grouped into maximal neighbouring clusters, and each cluster is a *crossing* c with an interval I_c^a on a's route and I_c^b on b's. The same physical place passed twice (into and out of a dead-end aisle) gives two crossings. An order $f \prec_c s$ is enforced per sample: s may be granted its sample $k \in I_c^s$ only after f has passed $\nu_c^s(k)$, the last sample of I_c^f that touches k. Crossing and head-on traffic therefore pass one at a time, while following traffic on the same lane follows at a gap.

Lock-freedom. Consider the graph whose edges are the route order of each truck plus $f \to s$ for every crossing. If it is acyclic and trucks stay within their grants, at least one unfinished truck can always advance, however many are stopped—the property behind temporal plan graphs and ADGs [8], [9]. We check it at every order change by *abstract execution*: ignoring time, every truck is advanced as far as the order conditions allow, repeatedly; the orders are lock-free iff all trucks reach their route ends. A crossing that contains a truck's route end (its parking slot) thereby always has that truck second.

D. Free-flow times and planned delay

For each truck, $\tau_n(g)$ is the time to reach arc length g if never blocked, from the stage speeds, an acceleration limit of 0.30 m/s^2 with stops at gear changes, and the lift/lower times. Given all orders, a *timed plan* pushes each second truck's entry behind its first truck's release, using only the entry and exit checkpoints of each interval. Its cost is

$$J = \sum_n w_n \big(T_n - T_n^{\text{free}}\big), \qquad w_n = 1 + \frac{\omega_n}{\alpha}, \tag{1}$$

where T_n and T_n^{free} are the planned and free-flow route-end times, ω_n is how long n has been waiting and $\alpha = 120$ s, so that long-waiting trucks are not starved.

E. Baseline: crossing time windows (A)

Our previous policy, used as the baseline, gives each crossing a token for its first truck only. A new crossing is ordered first-come in free-flow time. The token is issued when the first truck's planned entry is within $t_{\text{iss}} = 30$ s, with a deadline $t_{\text{slack}} = 10$ s after it; once the first truck's grant enters the crossing the order is fixed. An expired token is passed to the other truck if that lowers J and stays lock-free. Every second, a local search flips up to 30 open crossings, keeping a flip if it is lock-free and lowers J.

IV. Crossing Queues

Members, tickets and places. Each truck's interval in each crossing is a *member* with a *ticket*, initially its free-flow entry time; in each crossing the member with the lower ticket goes first, except that a truck already inside goes first. A truck's members that overlap on its route are one pass through one *place*, and the members of a place form one line, so a junction three routes meet at is one queue of three trucks. We do *not* merge a truck's members along a long shared lane: in development, merging made a 72 m member whose orders all froze the moment the truck entered it; trucks waited hundreds of seconds for trucks 25–70 m away, and one place held 29–37 members, almost the whole floor in one line.

Windows and tokens for every member. Unlike the baseline, every member gets its own token once its planned entry (from the timed plan) is within t_{iss}, with deadline $t_{\text{plan}} + t_{\text{slack}}$. A token only marks lateness; a member may enter early if its predecessor clears early. Holding a token or being inside fixes the member's orders.

Expiry and re-queuing. A member that has not entered by its deadline loses its token. If it is late only because its predecessor has not let it in, it keeps its place and receives a new window. Otherwise it rejoins at its new free-flow entry time: behind the other member of its crossing if that one is now due earlier, provided the resulting orders are lock-free; otherwise it keeps its place. The members behind it are then pulled forward by the next timed plan.

Pacing. With the timed plan, the manager knows when the predecessor at the next held place will release the sample a truck must wait at. It caps the truck's speed at

$$v_n = \frac{g_{\text{hold}} - p_n}{T_{\text{rel}} - t}, \tag{2}$$

so that it arrives as the place clears instead of stopping; no cap is given below 0.1 m/s (no creeping) or above the stage speed. Safety stays with the grant end.

Recovery from locks. New routes after a fault or spill are accepted even if they lock the orders (the old route is blocked). The manager then *heals*: it moves waiting members ahead of the members they wait behind, a few flips at a time, until the orders are lock-free again.

A. Global re-ordering by MILP

Every $T_{\text{opt}} = 5\,\text{s}$ the manager re-orders all *open* crossings (neither member holds a token or is in) at once. For each truck n, let $\mathcal{P}_n$ be the sorted checkpoints on its route where it may go second (the entry and exit samples of those intervals). The variables are delays $d_{n,j} \geq 0$ that apply from checkpoint j on, and a binary y_c per open crossing ($y_c = 1$: a first). For each crossing, each orientation (f, s) still possible and each checkpoint $\kappa \in \{\text{in}, \text{out}\}$ of I_c^s with release point $r = \nu_c^s(\kappa)$ on f's route,

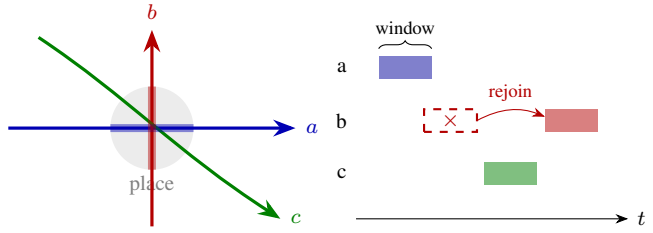
$$\tau_s(\kappa) + d_{s,j(\kappa)} \geq \tau_f(r) + d_{f,j(r)} - M\,(1 - \delta_{c,f}), \tag{3}$$

$$d_{n,j} \geq d_{n,j-1}, \tag{4}$$

with $\delta_{c,a} = y_c$, $\delta_{c,b} = 1 - y_c$ and $M = 10^4\,\text{s}$; $j(\cdot)$ picks the delay variable in force at that arc length. If r is the end of f's route, f parks there and $\tau_f(r)$ includes a 10 s parking time. Crossings already fixed enter (3) without the big-M term. The objective is

$$\min \sum_n w_n\, d_{n,|\mathcal{P}_n|} \;+\; \varepsilon \sum_{n,j} d_{n,j}, \qquad \varepsilon = 10^{-3}, \tag{5}$$

i.e., the weighted end delay of (1), plus a small term that keeps intermediate delays no larger than needed. We solve it with HiGHS through `scipy.optimize.milp` [26], [27] with a node limit of 2,000 instead of a wall-clock limit, so a run repeats exactly. Every open member's ticket is then set to its delayed entry time. The new orders are kept only if the abstract execution is lock-free, every fixed order is unchanged, and the cost J of the timed plan falls; otherwise the previous tickets are restored. Because (3) uses two checkpoints per interval, the exact sample-level lock check remains the gate. The local search of the baseline runs afterwards.



(a) three crossings $\{a, b\}$, $\{a, c\}$, $\{b, c\}$ form one place: one line

(b) b misses its window and rejoins behind c

Fig. 1. Crossing queues. (a) Members of the three trucks at one place form one line. (b) Each member's token opens a window $[t,\ t_{\text{plan}} + t_{\text{slack}}]$; a member that misses it rejoins at its new free-flow time if the orders stay lock-free, and the member behind is pulled forward.

Algorithm 1 Traffic manager period (every 1 s)

```
drop passed crossings; mark entered members
if orders locked (abstract execution) then
    heal: move waiting members ahead
for members past their deadline, not entered do
    if held by predecessor then
        renew window
    else
        rejoin at new free-flow ticket if lock-free
if global and T_opt elapsed then
    solve (3)–(5); set open tickets
    keep iff lock-free ∧ fixed orders kept ∧ J falls
local search: swap tickets of ≤ 30 open crossings if J falls
issue tokens to members due within t_iss
per truck: grant end from order conditions and claims; pace (2)
```

B. Physical time model (variant Q3p)

The variant Q3p replaces the free-flow times by a quasi-static model of the loaded truck. Link masses of the chassis model and the load on the forks give the center of gravity; longitudinal tip-over limits $a^- = g\,(x_f - x_G)/z_G$ and $a^+ = g\,(x_G - x_r)/z_G$ (front and rear axle positions x_f, x_r), lateral $g\,(b/2)/z_G$, traction μg, wheel torque and wheel speed bound a load-dependent speed profile. A standing truck adds its restart time, and a timed plan that makes a truck stop adds $v/(2a_{\text{brake}}) + v/(2a_{\text{up}})$, so heavy loads are less often stopped. The same profile drives the simulated vehicle in all policies. At the operating acceleration cap of 0.30 m/s^2 the physical limits bind only for loads above about 12.5 kg (of 5–13 kg sampled per job), so the expected effect is small.

TABLE I
POLICIES (ALL SHARE ROUTES, CLAIMS, TA/RA)

	Order mechanism	Pace	Re-order	ETA
A	time windows, first truck's token	–	local	simple
B	as A	–	local	physical
Q1	crossing queues	–	local	simple
Q2	crossing queues	yes	local	simple
Q3	crossing queues	yes	MILP + local	simple
Q3p	crossing queues	yes	MILP + local	physical
L2	critical sections [5], [7]	–	heuristic	–
L3	ADG, fixed order [9]	–	none	plan
L4	prioritized reservation [12], [21]	–	every 5 s	free + wait

TABLE II
EVALUATION, 117 CELLS (SEEDS 700–704, SUMS OVER CELLS)

	Jobs	>60 s	>300 s	Max (s)	Blocked	GL	Stops	RA fail
A	2,982	343	21	540	4.9 %	1	3,461	353
B	2,969	378	4	420	4.5 %	0	3,436	285
Q1	2,970	394	7	398	5.1 %	0	3,732	325
Q2	2,815	385	14	479	3.4 %	0	3,730	339
Q3	2,988	51	0	124	1.3 %	0	3,259	24
Q3p	2,971	34	0	120	1.3 %	0	3,310	16
L2	3,064	178	0	228	3.9 %	0	–	–
L3	2,749	794	49	931	8.5 %	2	–	–
L4	2,896	490	16	630	7.8 %	0	–	–

Waits are behind another truck. GL: gridlock cells. All policies: zero truck–truck overlaps, no wait cycle >60 s. –: not tabulated in our records.

V. EXPERIMENTAL SETUP

A. Floors, disturbances and simulation

A generator turns a seed into one of three floor types: a *rack warehouse* (hall 70–110×45–70 m, 2–4 rack blocks, picking aisles 6.8–8.0 m), a *process shop* (hall 60–100×50–80 m, column grid 10–14 m, machines of 3–9 m with input and output slots), and a *multi-hall* plant joining 2–3 such halls through 5–8 m doors and, for some seeds, a single-lane connector 4–4.5 m wide where oncoming trucks cannot pass. Six conditions are run: C0 (traffic only; 5, 10 or 20 trucks) and, with 10 trucks, C1 faults (a 30–120 s stop, a permanent breakdown and a 1–2 m spill blocking an aisle), C2 communication (0.5 s delay, 20 % grant loss), C3 localization (slowly varying bias up to 0.17 m), C4 pedestrians (10 people at 1.2 m/s; a truck stops when one is in its protective field, braking distance +1 m ahead), and C5 (C1–C4 together). A CPU simulator steps at 0.1 s for 1,800 s per cell. All policies use the same vehicle model (Sec. IV-A, acceleration cap 0.30 m/s^2) and loads drawn uniformly from 5–13 kg per job.

B. Policies

Table I lists the policies. Q1–Q3 add queues, pacing and global ordering in turn; B and Q3p use the physical time model. The literature methods are mapped onto the same routes and safety layer, so they decide only order and reservations: L2 (critical sections, DISTANCE heuristic: the truck closer to the section goes first, revised every second and changed only where the yielding truck can still stop and the new orders are acyclic [5], [7]), L3 (ADG: the free-flow order at route assignment, never changed [9]), and L4 (prioritized space–time reservation by job order on fixed routes, 30 s window re-solved every 5 s, executed in crossing order [12], [13], [21]). Tile reservation [2] (L1; 0.25 m tiles, whole-route banker's-style check) still had wait cycles over 60 s in 21 of 40 development cells after three fixes, so it is not compared here [TODO: add L1 evaluation result if completed]. SADG re-ordering [11] is closest to Q3 and was not reimplemented.

C. Protocol and metrics

Parameters ($t_{\text{iss}} = 30$ s, $t_{\text{slack}} = 10$ s, $\alpha = 120$ s, pace floor 0.1 m/s, $T_{\text{opt}} = 5$ s, node limit 2,000) were left at their initial values on development seeds 100–102. Two code fixes found there—unmerged members (Sec. IV) and keeping replanned routes clear of other trucks' claims—were made before the code was frozen. The evaluation seeds 700–704 give 120 cells (3 floor types × 5 seeds × 8 condition/fleet-size combinations), each run once per policy. One cell fails floor generation for every policy, and any cell a policy did not finish within 10 h of wall time is removed from all policies, leaving 117 cells. A policy passes if it has zero truck–truck overlaps (ground truth), no wait cycle longer than 60 s and no gridlock (no working truck advancing for 300 s). We report completed jobs, the number of waits behind another truck longer than 60 and 300 s, the longest such wait, the mean share of truck time spent blocked, stops, and failed RA attempts.

D. Large-scale study

To stress scale, floors are generated with the hall-size ranges ×1.5 (area ×2.25; e.g. rack seed 800: 149.1 × 77.2 m, 281 slots; multi-hall seed 800: 257.6 × 73.4 m, 244 slots) and run with 20 trucks. Jobs pick a random full slot and drop at a random empty one (60 % of slots initially full), and a truck takes its next job where it dropped the last one (*chaining*) instead of returning to a holding slot. Seeds 800–805, three floor types and conditions C0 and C5 give 36 cells per policy, with the evaluated settings unchanged and a larger planner budget for all policies. Cells over the 16 h limit were rerun with a parallel planner that returns the same plans (identical trajectories on a check cell), so all 36 cells are compared. Differences in jobs are reported as paired cell bootstrap intervals (4,000 resamples, 95 %) of the ratio of job totals.

VI. RESULTS

A. 117-cell evaluation

Table II gives the results. All nine policies kept trucks apart and none had a wait cycle over 60 s; A and L3 stopped in gridlock in one and two cells, L3's being waits on an immobile truck rather than cycles. All five queue and window variants that are judged (B, Q1–Q3p) passed.

Ablation. Queues alone (Q1) did not help: jobs within 0.4 % of A, but more long waits and stops. Pacing with local re-ordering (Q2) lowered jobs by 5.2 % against Q1. Global re-ordering is what changed the picture: Q3 completed as many

TABLE III
LARGE-SCALE STUDY, 36 CELLS (20 TRUCKS, 1.5× FLOORS, CHAINING)

	Jobs	>60 s	>300 s	Max (s)	Blocked	GL	Cyc>60
A	1,781	1,488	118	1,198	21.2 %	0	0
Q1	1,658	1,450	162	1,669	23.9 %	1	2
Q3	1,929	319	77	1,362	9.0 %	1	2
Q3p	1,971	368	63	1,509	8.4 %	1	3
L2	1,972	690	49	1,244	13.1 %	1	2
L4	1,791	1,075	59	1,337	18.9 %	1	1

Cyc>60: cells with a wait cycle over 60 s. Zero truck–truck overlaps in all runs. Jobs are counted at chaining and are not comparable with Table II.

[TODO: Figure: per-cell paired job difference Q3 − A and Q3 − L2 by floor type (36 cells), and wait-length distribution (117 cells). Data: result.json of eval700 and r2/r2x runs.]

Fig. 2. [TODO: Per-cell differences and wait distributions.]

jobs as A (+0.2 %) while waits over 60 s fell from 343 to 51 (−85 %), waits over 300 s from 21 to 0, the longest wait from 540 s to 124 s, the blocked share from 4.9 % to 1.3 %, stops by 5.8 % and failed RA attempts from 353 to 24. The physical time model did not raise throughput (B vs. A −0.4 %, Q3p vs. Q3 −0.6 %), though Q3p had fewer long waits (34 vs. 51). Throughput differences between A, B, Q1, Q3 and Q3p are within 0.7 %, and we did not test whether they exceed chance. By the prespecified rule (highest throughput among passing queue variants), Q3 was adopted.

Literature methods. The critical-section coordinator L2 completed the most jobs, 2.5 % more than Q3, but had 3.5 times as many waits over 60 s and a longest wait of 228 s. The fixed order of L3 had the most long waits (794; longest 931 s), and priority reservations (L4) made low-priority trucks wait long (490; 630 s). The price of Q3 is computation: the MILP took 29,931 s of wall time over the 117 cells, 256 s per 1,800 s simulated cell.

B. Large-scale study

Tables III and IV summarize the 20-truck floors. Q3, Q3p and L2 completed 8–11 % more jobs than A and are indistinguishable from each other overall (all three pairwise intervals contain zero). The gains over A come from the rack (Q3 +25.6 %) and multi-hall floors; on shop floors Q3, Q3p, L2 and A are within about 2 %. Q3 beat L2 on rack floors (+9.9 %) but trailed it on multi-hall floors (−13.2 %, interval containing zero). Waits over 60 s were 0.46 and 0.53 times those of L2 for Q3 and Q3p, but waits over 300 s were more frequent (77 and 63 vs. 49), mostly on multi-hall floors (53, 30 and 9).

Unlike the 117-cell evaluation, every policy except A had at least one gridlock or wait cycle over 60 s here (Table III). Two Q3 cells collapsed: multi_802_C5_20 gridlocked at 1,443 s with no job done (other policies 11–48), and multi_805_C5_20 finished 4 jobs (Q3p 48, L2 31). Both started within 1 s of a spill or fault, when 4–12 trucks replanned at once and the new routes locked the orders. Healing turns at most four orders per period and cannot turn crossings already entered; and since a candidate detour is tested against the whole order set, every detour is rejected while the orders are locked, which we believe is why RA did not help (289 and 329 failed RA attempts, against 11 for Q3p in the same cells). This diagnosis is from event logs and code; the cells were not rerun. Across all 216 runs, healing succeeded 3 times and failed 11,041 times. This mechanism is shared by every policy built on the crossing orders (A, Q1, Q3, Q3p, L2–L4) and also caused the L2 deadlock in rack_804_C5_20. Leaving the two cells out post hoc, Q3 − L2 becomes +1.5 % [−1.9, 5.3].

Computation. The MILP did not keep up with real time on the larger floors: per cell (one core of an AMD EPYC 9755 node) it took 439 s (shop), 10,170 s (rack) and 13,792 s (multi-hall) for 1,800 s simulated, with a maximum of 50,072 s. On the 21 cells every policy finished within the first 16 h, one solve took 1.2, 8.8 and 27.6 s on average on the three floor types, against the 5 s period, and only 34–36 % of solves were kept; the rest raised J or conflicted with fixed orders. In a 600 s profiling run on a rack floor, solves had on average 621 binaries and 5,136 rows, model building took about 0.08 s and the node limit did not bound the solve time. Restricting the binaries to crossings reached within a horizon of 60 or 120 s cut the time 6–20 fold but lost 3.6–7.1 % of jobs on those 21 cells, with intervals below zero, so we did not adopt it.

VII. DISCUSSION AND LIMITATIONS

Most of the benefit of crossing queues comes from the global ordering: queues and pacing alone did not help, and pacing with only local re-ordering hurt. The MILP mainly removes long waits rather than adding throughput at the original scale; at 20 trucks on 2.25 times the area it also adds throughput, matching a well-tuned critical-section heuristic with fewer waits over 60 s but more over 300 s.

The limitations are substantial. (i) The MILP is not real-time on large floors with the node-limited solver we used; horizon truncation was too costly in jobs, and how to bound the solve time without losing its benefit is open. (ii) Locked orders after simultaneous replanning are not reliably healed, which collapsed two Q3 cells and affects all crossing-order policies. (iii) All comparisons are in a CPU simulator with perfect path tracking; representative runs in NVIDIA Isaac Sim with physical vehicles were used only for inspection, since single runs differ by chance and pallets slipped off the forks in transport there (8–14 lowerings without a pallet per policy on a scaled rack floor), an issue also seen without chaining whose cause is still open. (iv) Each cell was run once; the 117-cell throughput differences were not tested for significance. (v) The vehicle parameters beyond the measured dimensions, load range, pedestrian behaviour and protective field are estimates or simple models, and truck–pedestrian contacts, recorded but

TABLE IV
PAIRED DIFFERENCES IN COMPLETED JOBS, LARGE-SCALE STUDY (% OF THE SECOND POLICY; CELL BOOTSTRAP, 4,000 RESAMPLES, 95 % INTERVAL)

	All 36	Shop 12	Rack 12	Multi-hall 12
Q3 − A	+8.3 [1.9, 15.7]	−2.1 [−4.6, 0.8]	+25.6 [13.1, 40.0]	+12.9 [−7.6, 31.8]
Q3 − Q1	+16.3 [9.4, 25.5]	+1.3 [−2.2, 5.5]	+40.2 [25.5, 59.6]	+26.8 [10.8, 46.8]
Q3 − L2	−2.2 [−8.8, 3.6]	−2.3 [−5.8, 1.4]	+9.9 [2.4, 20.2]	−13.2 [−36.0, 2.4]
Q3 − L4	+7.7 [2.0, 12.8]	+4.2 [1.1, 6.9]	+17.8 [10.2, 28.1]	+4.0 [−16.3, 19.4]
Q3p − Q3	+2.2 [−4.0, 11.5]	+0.6 [−1.9, 3.0]	−8.1 [−13.8, −2.6]	+17.3 [−5.0, 65.9]
Q3p − L2	−0.1 [−3.9, 4.0]	−1.7 [−4.8, 1.4]	+1.0 [−4.4, 9.5]	+1.8 [−9.8, 13.5]
L2 − A	+10.7 [5.0, 18.4]	+0.2 [−3.7, 3.7]	+14.3 [3.9, 24.2]	+30.1 [13.2, 58.0]

not judged, occurred under all policies. (vi) Literature methods were adapted to fixed continuous routes and our safety layer; their published forms assume grids or roadmaps.

VIII. CONCLUSION

Crossing queues give every truck at every place where free-form routes touch its own ticket, window and token, re-queue late trucks where the order graph stays acyclic, and re-order all open crossings by a MILP. In 117 generated cells they kept throughput equal to a time-window baseline while cutting waits over 60 s by 85 %, without overlaps or deadlocks, and on larger floors with 20 trucks they matched an adapted critical-section coordinator in throughput. Bounding the solve time and healing locked orders after mass replanning are the next steps [TODO: future work as decided].

REFERENCES

[1] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," *Int. J. Robot. Res.*, vol. 29, no. 5, pp. 485–501, 2010.

[2] K. J. C. Fransen, M. A. Reniers, and J. A. W. M. van Eekelen, "Deadlock avoidance algorithm for AGVs on a tessellated layout," in *Proc. IEEE 18th Int. Conf. Automation Science and Engineering (CASE)*, 2022, pp. 1163–1169.

[3] F. Pratissoli, R. Brugioni, N. Battilani, and L. Sabattini, "Hierarchical traffic management of multi-AGV systems with deadlock prevention applied to industrial environments," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 3155–3169, 2024.

[4] P. Verma, J. M. Olm, and R. Suárez, "Traffic management of multi-AGV systems by improved dynamic resource reservation," *IEEE Access*, vol. 12, 2024.

[5] F. Pecora, H. Andreasson, M. Mansouri, and V. Petkov, "A loosely-coupled approach for multi-robot coordination, motion planning and control," in *Proc. Int. Conf. Automated Planning and Scheduling (ICAPS)*, vol. 28, 2018, pp. 485–493.

[6] A. Mannucci, L. Pallottino, and F. Pecora, "Provably safe multi-robot coordination with unreliable communication," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 3232–3239, 2019.

[7] ——, "On provably safe and live multirobot coordination with online goal posting," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1973–1991, 2021.

[8] W. Hönig, T. K. S. Kumar, L. Cohen, H. Ma, H. Xu, N. Ayanian, and S. Koenig, "Multi-agent path finding with kinematic constraints," in *Proc. Int. Conf. Automated Planning and Scheduling (ICAPS)*, 2016.

[9] W. Hönig, S. Kiesel, A. Tinka, J. W. Durham, and N. Ayanian, "Persistent and robust execution of MAPF schedules in warehouses," *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 1125–1131, 2019.

[10] M. Čáp, J. Grégoire, and E. Frazzoli, "Provably safe and deadlock-free execution of multi-robot plans under delaying disturbances," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2016.

[11] A. Berndt, N. van Duijkeren, L. Palmieri, A. Kleiner, and T. Keviczky, "Receding horizon re-ordering of multi-agent execution schedules," *IEEE Trans. Robot.*, vol. 40, pp. 1356–1372, 2024.

[12] J. Li, A. Tinka, S. Kiesel, J. W. Durham, T. K. S. Kumar, and S. Koenig, "Lifelong multi-agent path finding in large-scale warehouses," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 13, 2021, pp. 11 272–11 281.

[13] S. Varambally, J. Li, and S. Koenig, "Which MAPF model works best for automated warehousing?" in *Proc. Int. Symp. Combinatorial Search (SoCS)*, vol. 15, 2022, pp. 190–198.

[14] S. Bonetti, S. Proia, S. Guidetti, and L. Sabattini, "AGV traffic management in automated industrial plants: An enhanced lifelong multi-agent path finding approach," in *Proc. IEEE Int. Conf. Automation Science and Engineering (CASE)*, 2024, pp. 626–632.

[15] V. Digani, L. Sabattini, C. Secchi, and C. Fantuzzi, "Ensemble coordination approach in multi-AGV systems applied to industrial warehouses," *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 3, pp. 922–934, 2015.

[16] S. A. Reveliotis, "Preservation of traffic liveness in MPC schemes for guidepath-based transport systems," in *Proc. IEEE Int. Conf. Automation Science and Engineering (CASE)*, 2018.

[17] ——, "An MPC scheme for traffic coordination in open and irreversible, zone-controlled, guidepath-based transport systems," in *Proc. IEEE Int. Conf. Automation Science and Engineering (CASE)*, 2019, pp. 1782–1788.

[18] A. Mascis and D. Pacciarelli, "Job-shop scheduling with blocking and no-wait constraints," *Eur. J. Oper. Res.*, vol. 143, no. 3, pp. 498–517, 2002.

[19] A. D'Ariano, D. Pacciarelli, and M. Pranzo, "A branch and bound algorithm for scheduling trains in a railway network," *Eur. J. Oper. Res.*, vol. 183, no. 2, pp. 643–657, 2007.

[20] M. Phillips and M. Likhachev, "SIPP: Safe interval path planning for dynamic environments," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2011.

[21] M. Čáp, P. Novák, A. Kleiner, and M. Selecký, "Prioritized planning algorithms for trajectory coordination of multiple mobile robots," *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 3, 2015.

[22] J. Peng and S. Akella, "Coordinating multiple robots with kinodynamic constraints along specified paths," *Int. J. Robot. Res.*, vol. 24, no. 4, 2005.

[23] A. Maoudj, S. Mekid, and A. L. Christensen, "Lifelong MAPF for shipment preparation in stochastic warehouses," in *Proc. IEEE Int. Conf. Automation Science and Engineering (CASE)*, 2024, pp. 1006–1011.

[24] Popolizio, Vinetti, Combrink, Roselli, Fanti, and Fabian, "Online conflict-free scheduling of fleets of autonomous mobile robots," in *Proc. IEEE Int. Conf. Automation Science and Engineering (CASE)*, 2024, pp. 3063–3068.

[25] S. Dergachev and K. Yakovlev, "Distributed multi-agent navigation based on reciprocal collision avoidance and locally confined multi-agent path finding," in *Proc. IEEE Int. Conf. Automation Science and Engineering (CASE)*, 2021, pp. 1489–1494.

[26] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Math. Program. Comput.*, vol. 10, no. 1, pp. 119–142, 2018.

[27] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.